

lab1

September 21, 2026

1 № 1

1.1

4 : (26) : 2 + 3 . :

1.1.1

Jupyter Notebook, pandas — ,
, . : , ,
, 3 4.

1.1.2

20171129_FINAL_FINAL.xlsx, Sheet1 — 28 — 22 2017 ,
, , . :

1.1.3

1 ,
2
3 : , ,
4 ,
5 , — +
6
7
8

1.1.4

1. `.ipynb` (Kernel → Restart & Run All) —
2. `master_hourly.csv` —
3. HTML PDF.
4. (8).

1.1.5

0	,
1	,
2	, , , ,
4	, ,

:	, , , ,
---	---------

1.2 Jupyter

- Shift+Enter , Esc A / Esc B / , Esc M (Markdown), Esc Y —
- Kernel → Restart & Run All —
- ,

1.3 0.

```
[1]: = ' ' # ' '
     = ' -32' # ' -31'
     = 1 #

assert and and , ' , '

[2]: # --- : -----
     = ['P>1', 'P>5', 'P>10', 'P>30', 'P>50', 'P>100', 'E>0.8', 'E>2.0']

     = [(' ', ' '),
```


Because xelatex was not found in the package registry and your project depends on xelatex, we can conclude that your project's requirements are unsatisfiable.

hint: While the active Python version is 3.13, the resolution failed for other Python versions supported by your project. Consider limiting your project's supported Python versions using ``requires-python``.
help: If you want to add the package regardless of the failed resolution, provide the ``--frozen`` flag to skip locking and syncing.

```
[4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

pd.set_option('display.width', 200)
pd.set_option('display.max_columns', 40)
plt.rcParams.update({'figure.dpi': 110, 'axes.spines.top': False,
                    'axes.spines.right': False, 'font.size': 9})

= '20171129_FINAL_FINAL.xlsx'
```

2 1.

2.0.1 1.1

Sheet1 — pd.read_excel() —

```
[5]: #
pd.read_excel( )
```

```
[5]: Unnamed: 0 ftp://ftp.swpc.noaa.gov/pub/lists/particle/ Unnamed: 2
Unnamed: 3 Unnamed: 4 Unnamed: 5 Unnamed: 6 Unnamed: 7 Unnamed: 8 Unnamed: 9
Unnamed: 10 Unnamed: 11 Unnamed: 12 \
0 NaN NaN NaN NaN NaN NaN NaN
NaN NaN NaN NaN NaN NaN NaN
NaN NaN NaN
1 NaN :Data_list: 20170901_Gs_part_5m.txt NaN
```

NaN	NaN	NaN	NaN	NaN	NaN	NaN	
NaN	NaN	NaN					
2	NaN				:Created: 2017 Sep 02 0011 UTC		NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	
NaN	NaN	NaN					
3	NaN	# Prepared by the U.S. Dept. of Commerce, NOAA...					NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	
NaN	NaN	NaN					
4	NaN	# Please send comments and suggestions to SWPC...					NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN	
NaN	NaN	NaN					
...	...				...	...	
...	...	...	...	...	...	...	
...	...	...					
7220	NaN					2017	9
21	2335	58017	84900	21.9	0.361	0.202	0.084
0.0717	0.0475						
7221	NaN					2017	9
21	2340	58017	85200	14.1	0.343	0.309	0.118
0.0679	0.0328						
7222	NaN					2017	9
21	2345	58017	85500	27.1	0.465	0.237	0.106
0.0593	0.0251						
7223	NaN					2017	9
21	2350	58017	85800	23.7	0.36	0.205	0.127
0.091	0.0363						
7224	NaN					2017	9
21	2355	58017	86100	32.4	0.377	0.166	0.106
0.0593	0.0251						

Unnamed: 13 Unnamed: 14 Unnamed: 15 Unnamed: 16 Unnamed: 17 Unnamed: 18						
Unnamed: 19 Unnamed: 20 Unnamed: 21 Unnamed: 22 Unnamed: 23 Unnamed: 24						
Unnamed: 25 \						
0	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
1	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
2	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
3	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
4	NaN	NaN	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	

NaN						
...	...	...	...	...	...	...
...	...	...	...	...	...	...
...						
7220	143000	32900	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
7221	143000	32100	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
7222	140000	32500	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
7223	140000	32700	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						
7224	143000	33900	NaN	NaN	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	
NaN						

<http://umtof.umd.edu/pm/crn/> Unnamed: 27 Unnamed: 28

Unnamed: 29 Unnamed: 30 Unnamed: 31

0				NaN	NaN	NaN
NaN	NaN	NaN				
1			SOHO CELIAS Proton Monitor		NaN	NaN
NaN	NaN	NaN				
2	Solar Wind Parameters for Carrington Rotation ...				NaN	NaN
NaN	NaN	NaN				
3			Proton speed [km/sec]		NaN	NaN
NaN	NaN	NaN				
4	Proton density [protons per cubic centimeter]				NaN	NaN
NaN	NaN	NaN				
...				...	...	...
...	...	...				
7220				NaN	NaN	NaN
NaN	NaN	NaN				
7221				NaN	NaN	NaN
NaN	NaN	NaN				
7222				NaN	NaN	NaN
NaN	NaN	NaN				
7223				NaN	NaN	NaN
NaN	NaN	NaN				
7224				NaN	NaN	NaN
NaN	NaN	NaN				

[7225 rows x 32 columns]

1.1: ()

```

,
,
header.

```

2.0.2 1.2

```

,
: pd.read_excel( , header=None).

```

30

```

[6]: #
RAW = pd.read_excel( , header=None)

print(len(RAW))
RAW[:30]

```

7226

```

[6]:      0      1      2      3      4
5      6      7      8      9     10     11     12     13     14 15 \
0  NaN      ftp://ftp.swpc.noaa.gov/pub/lists/particle/  NaN  NaN  NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
1  NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
2  NaN      :Data_list: 20170901_Gs_part_5m.txt  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
3  NaN      :Created: 2017 Sep 02 0011 UTC  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
4  NaN  # Prepared by the U.S. Dept. of Commerce, NOAA...  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
5  NaN  # Please send comments and suggestions to SWPC...  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
6  NaN      #      NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
7  NaN      # Label: P > 1 = Particles at >1 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
8  NaN      # Label: P > 5 = Particles at >5 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
9  NaN      # Label: P >10 = Particles at >10 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
10 NaN      # Label: P >30 = Particles at >30 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
11 NaN      # Label: P >50 = Particles at >50 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
12 NaN      # Label: P>100 = Particles at >100 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
13 NaN      # Label: E>0.8 = Electrons at >0.8 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN
14 NaN      # Label: E>2.0 = Electrons at >2.0 Mev  NaN  NaN      NaN
NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN  NaN

```

15	NaN				# Label: E>4.0 = Electrons at >4.0 Mev	NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
16	NaN				# Units: Particles = Protons/cm2-s-sr	NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
17	NaN				# Units: Electrons = Electrons/cm2-s-sr	NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
18	NaN				# Source: GOES-15	NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
19	NaN				# Location: W135	NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
20	NaN				# Missing data: -1.00e+05	NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
21	NaN					NaN	NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
22	NaN	5-minute	GOES-15	Solar Particle and Electron ...		NaN	NaN	NaN		
NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
23	NaN					NaN	NaN	NaN	NaN	
Modified	Seconds		NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
24	NaN					# UT	C	D	ate	Time
Julian	of the		NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25	NaN					# YR	MO	DA	HHMM	
Day	Day	P > 1	P > 5	P > 10	P > 30	P > 50	P > 100	E > 0.8	E > 2.0	NaN
26	NaN						2017	8	28	0
57993		0	12	0.26	0.18	0.114	0.104	0.0793	23600	1180 NaN
27	NaN						2017	8	28	5
57993		300	19.4	0.69	0.314	0.112	0.0612	0.0251	23100	1160 NaN
28	NaN						2017	8	28	10
57993		600	8.48	0.168	0.13	0.0717	0.0614	0.0304	22200	1090 NaN
29	NaN						2017	8	28	15
57993		900	15.4	0.566	0.198	0.0926	0.0568	0.0251	20400	997 NaN

							16	17	18	
19	20	21	22	23	24	25				26
27	28	29		30		31				
0							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN	http://umtof.umd.edu/pm/crn/			
NaN	NaN	NaN		NaN		NaN				
1							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN				NaN
NaN	NaN	NaN		NaN		NaN				
2							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN	SOHO CELIAS Proton Monitor			
NaN	NaN	NaN		NaN		NaN				
3							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN	Solar Wind Parameters for Carrington Rotation ...			
NaN	NaN	NaN		NaN		NaN				
4							NaN	NaN	NaN	

9

20							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN				NaN
NaN	NaN	NaN		NaN		NaN				
21							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN				NaN
NaN	NaN	NaN		NaN		NaN				
22							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN				NaN
NaN	NaN	NaN		NaN		NaN				
23								Quarterly Daily Solar Data	NaN	NaN
NaN	NaN	NaN	NaN	NaN	NaN	NaN				NaN
NaN	NaN	NaN		NaN		NaN				
24							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN				NaN
NaN	NaN	NaN		NaN		NaN				
25							NaN	NaN	NaN	
NaN	NaN	NaN	NaN	NaN	NaN	NaN				NaN
NaN	NaN	NaN		NaN		NaN				
26							NaN	Month	Date	Radio Flux
10.7	NaN	NaN	NaN	NaN	NaN	NaN				#
YR	MO	DA	HH	SPEED	DENSITY					
27							2017	8	28	
82	NaN	NaN	NaN	NaN	0.0	0.0				17
8	28	0	285			3				
28							2017	8	29	
84	NaN	NaN	NaN	NaN	1.0	1.0				17
8	28	1	305			3.4				
29							2017	8	30	
87	NaN	NaN	NaN	NaN	2.0	2.0				17
8	28	2	309			1.7				

2.0.3 1.3

```
, (RAW.notna().sum()),
: 32 . , , ,
: ?
```

```
[7]: #
RAW.notna().sum()
```

```
[7]: 0 0
1 7223
2 7202
3 7202
4 7202
5 7203
```

```

6      7203
7      7198
8      7198
9      7198
10     7198
11     7198
12     7198
13     7198
14     7198
15        0
16       32
17       26
18       26
19       26
20        0
21        0
22        0
23        0
24     601
25     601
26     608
27     602
28     602
29     602
30     602
31     602
dtype: int64

```

1.3: ()

3 , () 1-14, 16-19, 24-31

2.0.4 1.4

```

[8]: # A
RAW.iloc[:30, 1:14]

```

```

[8]:
6      7      8      9      10      11      12      13      1      2      3      4      5
0      ftp://ftp.swpc.noaa.gov/pub/lists/particle/ NaN NaN NaN NaN
NaN NaN NaN NaN NaN NaN NaN NaN
1      NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN NaN
NaN NaN NaN NaN NaN NaN NaN NaN
2      :Data_list: 20170901_Gs_part_5m.txt NaN NaN NaN NaN

```

3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
8	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
10	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
11	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
12	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
13	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
14	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
15	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
16	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
17	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
18	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
19	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
21	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
22	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
23	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
24	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
25	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
Day	P > 1	P > 5	P >10	P >30	P >50	P>100	E>0.8				

26							2017	8	28	0	57993
0	12	0.26	0.18	0.114	0.104	0.0793	23600				
27							2017	8	28	5	57993
300	19.4	0.69	0.314	0.112	0.0612	0.0251	23100				
28							2017	8	28	10	57993
600	8.48	0.168	0.13	0.0717	0.0614	0.0304	22200				
29							2017	8	28	15	57993
900	15.4	0.566	0.198	0.0926	0.0568	0.0251	20400				

```
[9]: # B
RAW.iloc[:30, 16:20]
```

```
[9]:
```

	16	17	18
19			
0	NaN	NaN	NaN
NaN			
1	NaN	NaN	NaN
NaN			
2	NaN	NaN	NaN
NaN			
3	NaN	NaN	NaN
NaN			
4	NaN	NaN	NaN
NaN			
5	NaN	NaN	NaN
NaN			
6	NaN	NaN	NaN
NaN			
7	NaN	NaN	NaN
NaN			
8	NaN	NaN	NaN
NaN			
9	NaN	NaN	NaN
NaN			
10	NaN	NaN	NaN
NaN			
11	NaN	NaN	NaN
NaN			
12	NaN	NaN	NaN
NaN			
13	ftp://ftp.swpc.noaa.gov/pub/indices/old_indice...	NaN	NaN
NaN			
14	NaN	NaN	NaN
NaN			
15	Product: Daily Solar Data	quar_DSD.txt	NaN
NaN			
16		:Issued: 0825 UT 04 Oct 2017	NaN

```

NaN
17                                     #      NaN      NaN
NaN
18 # Prepared by the U.S. Dept. of Commerce, NOAA...      NaN      NaN
NaN
19 # Please send comments and suggestions to SWP...      NaN      NaN
NaN
20                                     NaN      NaN      NaN
NaN
21                                     NaN      NaN      NaN
NaN
22                                     NaN      NaN      NaN
NaN
23                               Quarterly Daily Solar Data      NaN      NaN
NaN
24                                     NaN      NaN      NaN
NaN
25                                     NaN      NaN      NaN
NaN
26                                     NaN      Month      Date      Radio Flux
10.7
27                                     2017          8      28
82
28                                     2017          8      29
84
29                                     2017          8      30
87

```

```

[10]: # C
      RAW.iloc[:30, 24:32]

```

```

[10]:      24      25                                     26      27      28      29
30      31
0  NaN  NaN                                     http://umtof.umd.edu/pm/crn/  NaN  NaN  NaN
NaN  NaN
1  NaN  NaN                                     NaN  NaN  NaN  NaN
NaN  NaN
2  NaN  NaN                               SOHO CELIAS Proton Monitor  NaN  NaN  NaN
NaN  NaN
3  NaN  NaN  Solar Wind Parameters for Carrington Rotation ...  NaN  NaN  NaN
NaN  NaN
4  NaN  NaN                               Proton speed [km/sec]  NaN  NaN  NaN
NaN  NaN
5  NaN  NaN      Proton density [protons per cubic centimeter]  NaN  NaN  NaN
NaN  NaN
6  NaN  NaN                                     NaN  NaN  NaN  NaN
NaN  NaN

```

7	NaN	NaN	Measurement time (Year, Month, Day of Month, D...	NaN	NaN	NaN	
	NaN	NaN					
8	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
9	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
10	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
11	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
12	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
13	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
14	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
15	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
16	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
17	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
18	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
19	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
20	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
21	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
22	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
23	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
24	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
25	NaN	NaN		NaN	NaN	NaN	NaN
	NaN	NaN					
26	NaN	NaN		#	YR	MO	DA
	SPEED	DENSITY					HH
27	0.0	0.0		17		8	28
	285	3					0
28	1.0	1.0		17		8	28
	305	3.4					1
29	2.0	2.0		17		8	28
	309	1.7					2

1.4:

A	ftp://ftp.swpc.noaa.gov/pub/lists/Protons/20170901_Gs_part_5m.txt
B	ftp://ftp.swpc.noaa.gov/pub/indices/20170901_Rea/hold/ind10:7(2017)Q3_DSD.txt
C	http://umtof.umd.edu/pm/crn/ Proton speed [km/sec], Proton density [protons/cm ³]

3 2.

.	—	:	=
A	B-O	GOES-15:	5
B	Q-T	F10.7	
C	Y-AF		

3.0.1 2.1

DataFrame — A, B C.
(pd.to_numeric(..., errors='coerce')).

```
[11]: #
A = RAW.iloc[26:, 1:15]
B = RAW.iloc[27:, 16:20]
C = RAW.iloc[27:, 26:32]
A = A.loc[:A.last_valid_index()].convert_dtypes()
B = B.loc[:B.last_valid_index()].convert_dtypes()
C = C.loc[:C.last_valid_index()].convert_dtypes()

A.columns = RAW.iloc[23:26, 1:15].fillna("").astype(str).agg(' '.join, axis=0)
A.columns = ["year", "month", "day", "HHMM"] + [i.lstrip() for i in list(A.
↪columns[4:])]
B.columns = ["year"] + list(RAW.iloc[26, 17:20])
C.columns = ["year", "month", "day", "hour"] + list(RAW.iloc[26, 30:32])
```

3.0.2 2.2

```
- . ts datetime.
,
.
: - 2017 . — ,
, .
```

```
[12]: #
A["hour"] = A['HHMM'] // 100
A["minute"] = A['HHMM'] % 100
A["ts"] = pd.to_datetime(A[['year', 'month', 'day', 'hour', 'minute']])

B["ts"] = pd.to_datetime(B[['year', 'Month', 'Date']].rename(
    columns={
        'Month': 'month',
        'Date': 'day'
    }))

C.loc[C["year"] < 100, "year"] += 2000
C["ts"] = pd.to_datetime(C[['year', 'month', 'day', 'hour']])

for n, i in (('A', A), ('B', B), ('C', C)):
    print(f'Table {n}: min {min(i["ts"])}, max {max(i["ts"])}')
```

Table A: min 2017-08-28 00:00:00, max 2017-09-21 23:55:00

Table B: min 2017-08-28 00:00:00, max 2017-09-21 00:00:00

Table C: min 2017-08-28 00:00:00, max 2017-09-22 00:00:00

2.

```
[13]: assert len(A) == 7200, 'A 7200, %d' % len(A)
assert len(B) == 25, 'B 25, %d' % len(B)
assert len(C) == 601, 'C 601, %d' % len(C)
for , i in (('A', A), ('B', B), ('C', C)):
    = pd.to_datetime(['ts']).dt.year
    assert .min() == 2017 and .max() == 2017, ' %s 2017 - %s'
    print(' 2')
```

2

4 3.

4.0.1 3.1

(describe()).

```
[14]: #
for n, i in (('A', A), ('B', B), ('C', C)):
    print(f'=== {n} ===')
    print(f'Length = {len(i)}')
    print('Dtypes:')
    print(i.dtypes)
```

```

print('Count of NAs per col:')
print(i.isna().sum())
print(f'Count of full duplicates: {i.duplicated().sum()}')
print('Describe:')
print(i.describe())

```

=== A ===

Length = 7200

Dtypes:

year	Int64
month	Int64
day	Int64
HHMM	Int64
Modified Julian Day	Int64
Seconds of the Day	Int64
P > 1	Float64
P > 5	Float64
P >10	Float64
P >30	Float64
P >50	Float64
P>100	Float64
E>0.8	Float64
E>2.0	Float64
hour	Int64
minute	Int64
ts	datetime64[us]

dtype: object

Count of NAs per col:

year	0
month	0
day	0
HHMM	0
Modified Julian Day	0
Seconds of the Day	0
P > 1	3
P > 5	3
P >10	3
P >30	3
P >50	3
P>100	3
E>0.8	3
E>2.0	3
hour	0
minute	0
ts	0

dtype: int64

Count of full duplicates: 0

Describe:

	year	month	day	HHMM	Modified Julian Day	Seconds of
the Day	P > 1	P > 5	P >10	P >30	P >50	P>100
E>0.8	E>2.0	hour \				
count	7200.0	7200.0	7200.0	7200.0	7200.0	
7200.0	7197.0	7197.0	7197.0	7197.0	7197.0	7197.0
7197.0	7197.0	7200.0				
mean	2017.0	8.84	13.96	1177.5	58005.0	
43050.0	689.325064	156.626057	76.47402	18.838604	7.44684	1.65929
81862.741801	11401.502123	11.5				
min	2017.0	8.0	1.0	0.0	57993.0	
0.0	0.429	0.151	0.117	0.0576	0.0455	0.0224
4.3	0.133	0.0				
25%	2017.0	9.0	7.0	588.75	57999.0	
21525.0	11.7	0.279	0.186	0.0984	0.0597	0.0263
22200.0	552.0	5.75				
50%	2017.0	9.0	13.0	1177.5	58005.0	
43050.0	55.4	0.632	0.376	0.146	0.0866	0.0363
68600.0	4290.0	11.5				
75%	2017.0	9.0	19.0	1766.25	58011.0	
64575.0	560.0	107.0	35.8	0.99	0.249	0.067
143000.0	14100.0	17.25				
max	2017.0	9.0	31.0	2355.0	58017.0	
86100.0	32500.0	2890.0	1330.0	399.0	191.0	61.5
259000.0	110000.0	23.0				
std	0.0	0.366632	8.775483	692.481902	7.211603	
24943.113498	1625.968447	390.921861	222.362627	67.265595	28.620524	7.15889
65728.005824	16920.617446	6.922667				

	minute	ts
count	7200.0	7200
mean	27.5	2017-09-09 11:57:30
min	0.0	2017-08-28 00:00:00
25%	13.75	2017-09-03 05:58:45
50%	27.5	2017-09-09 11:57:30
75%	41.25	2017-09-15 17:56:15
max	55.0	2017-09-21 23:55:00
std	17.261461	NaN

=== B ===

Length = 25

Dtypes:

year	Int64
Month	Int64
Date	Int64
Radio Flux 10.7	Int64
ts	datetime64[us]

dtype: object

Count of NAs per col:

year	0
------	---

```

Month          0
Date           0
Radio Flux 10.7 0
ts             0

```

```
dtype: int64
```

```
Count of full duplicates: 0
```

```
Describe:
```

	year	Month	Date	Radio Flux 10.7	ts
count	25.0	25.0	25.0	25.0	25
mean	2017.0	8.84	13.96	92.68	2017-09-09 00:00:00
min	2017.0	8.0	1.0	71.0	2017-08-28 00:00:00
25%	2017.0	9.0	7.0	74.0	2017-09-03 00:00:00
50%	2017.0	9.0	13.0	84.0	2017-09-09 00:00:00
75%	2017.0	9.0	19.0	107.0	2017-09-15 00:00:00
max	2017.0	9.0	31.0	140.0	2017-09-21 00:00:00
std	0.0	0.374166	8.955817	22.206831	NaN

```
=== C ===
```

```
Length = 601
```

```
Dtypes:
```

```

year          Int64
month         Int64
day           Int64
hour          Int64
SPEED         Int64
DENSITY       Float64
ts            datetime64[us]

```

```
dtype: object
```

```
Count of NAs per col:
```

```

year          0
month         0
day           0
hour          0
SPEED         0
DENSITY       0
ts            0

```

```
dtype: int64
```

```
Count of full duplicates: 0
```

```
Describe:
```

	year	month	day	hour	SPEED	DENSITY	ts
count	601.0	601.0	601.0	601.0	601.0	601.0	601
mean	2017.0	8.840266	13.973378	11.480865	506.547421	2.907654	2017-09-09 12:00:00
min	2017.0	8.0	1.0	0.0	-1.0	-1.0	2017-08-28 00:00:00
25%	2017.0	9.0	7.0	5.0	447.0	1.8	2017-09-03 06:00:00

50%	2017.0	9.0	13.0	11.0	517.0	2.4	2017-09-09 12:00:00
75%	2017.0	9.0	19.0	17.0	584.0	3.4	2017-09-15 18:00:00
max	2017.0	9.0	31.0	23.0	806.0	31.7	2017-09-22 00:00:00
std	0.0	0.366664	8.781	6.938063	125.073291	2.637873	
NaN							

4.0.2 3.2 —

```

min      describe()      C.
—
1.4:      ,      ,      ,      ,      ,
NaN      ,
:      « » « »

```

```

[15]: #
print('=== BEFORE ===')
print('Describe:')
print(C.describe())
print('Speed negative:')
print(C.loc[C['SPEED'] < 0])
print(f'Count: {len(C.loc[C['SPEED'] < 0])}')
print('Density negative:')
print(C.loc[C['DENSITY'] < 0])
print(f'Count: {len(C.loc[C['DENSITY'] < 0])}')

C['SPEED'] = C['SPEED'].mask(C['SPEED'] < 0)
C['DENSITY'] = C['DENSITY'].mask(C['DENSITY'] < 0)

print('=== AFTER ===')
print('Describe:')
print(C.describe())
print('Speed negative:')
print(C.loc[C['SPEED'] < 0])
print(f'Count: {len(C.loc[C['SPEED'] < 0])}')
print('Density negative:')
print(C.loc[C['DENSITY'] < 0])
print(f'Count: {len(C.loc[C['DENSITY'] < 0])}')

```

=== BEFORE ===

Describe:

	year	month	day	hour	SPEED	DENSITY
ts						
count	601.0	601.0	601.0	601.0	601.0	601.0
601						

```

mean    2017.0  8.840266  13.973378  11.480865  506.547421  2.907654  2017-09-09
12:00:00
min      2017.0      8.0      1.0      0.0      -1.0      -1.0  2017-08-28
00:00:00
25%      2017.0      9.0      7.0      5.0      447.0      1.8  2017-09-03
06:00:00
50%      2017.0      9.0     13.0     11.0     517.0      2.4  2017-09-09
12:00:00
75%      2017.0      9.0     19.0     17.0     584.0      3.4  2017-09-15
18:00:00
max      2017.0      9.0     31.0     23.0     806.0     31.7  2017-09-22
00:00:00
std          0.0  0.366664      8.781   6.938063  125.073291  2.637873
NaN

```

Speed negative:

	year	month	day	hour	SPEED	DENSITY	ts
202	2017	9	4	7	-1	-1.0	2017-09-04 07:00:00
203	2017	9	4	8	-1	-1.0	2017-09-04 08:00:00
204	2017	9	4	9	-1	-1.0	2017-09-04 09:00:00
205	2017	9	4	10	-1	-1.0	2017-09-04 10:00:00
206	2017	9	4	11	-1	-1.0	2017-09-04 11:00:00
207	2017	9	4	12	-1	-1.0	2017-09-04 12:00:00
208	2017	9	4	13	-1	-1.0	2017-09-04 13:00:00
209	2017	9	4	14	-1	-1.0	2017-09-04 14:00:00

Count: 8

Density negative:

	year	month	day	hour	SPEED	DENSITY	ts
202	2017	9	4	7	-1	-1.0	2017-09-04 07:00:00
203	2017	9	4	8	-1	-1.0	2017-09-04 08:00:00
204	2017	9	4	9	-1	-1.0	2017-09-04 09:00:00
205	2017	9	4	10	-1	-1.0	2017-09-04 10:00:00
206	2017	9	4	11	-1	-1.0	2017-09-04 11:00:00
207	2017	9	4	12	-1	-1.0	2017-09-04 12:00:00
208	2017	9	4	13	-1	-1.0	2017-09-04 13:00:00
209	2017	9	4	14	-1	-1.0	2017-09-04 14:00:00

Count: 8

=== AFTER ===

Describe:

	year	month	day	hour	SPEED	DENSITY	ts
count	601.0	601.0	601.0	601.0	593.0	593.0	
mean	2017.0	8.840266	13.973378	11.480865	513.394604	2.960371	2017-09-09 12:00:00
min	2017.0	8.0	1.0	0.0	277.0	0.1	2017-08-28 00:00:00
25%	2017.0	9.0	7.0	5.0	449.0	1.8	2017-09-03 06:00:00

```

50%    2017.0      9.0      13.0      11.0      520.0      2.5  2017-09-09
12:00:00
75%    2017.0      9.0      19.0      17.0      585.0      3.4  2017-09-15
18:00:00
max     2017.0      9.0      31.0      23.0      806.0      31.7  2017-09-22
00:00:00
std      0.0  0.366664      8.781  6.938063  111.025317  2.615965
NaN
Speed negative:
Empty DataFrame
Columns: [year, month, day, hour, SPEED, DENSITY, ts]
Index: []
Count: 0
Density negative:
Empty DataFrame
Columns: [year, month, day, hour, SPEED, DENSITY, ts]
Index: []
Count: 0

```

```

3.2: (
      8,
      (4 7 14 )
      -1,
      ,

```

4.0.3 3.3 —

```

A , describe()
,
:
« » —
3.

```

```

[16]: #
A.describe()

```

```

[16]:      year      month      day      HHMM  Modified Julian Day  Seconds of
the Day      P > 1      P > 5      P >10      P >30      P >50      P>100
E>0.8      E>2.0      hour \
count  7200.0    7200.0    7200.0    7200.0          7200.0
7200.0    7197.0    7197.0    7197.0    7197.0    7197.0    7197.0
7197.0    7197.0    7200.0
mean   2017.0      8.84     13.96     1177.5          58005.0
43050.0  689.325064  156.626057   76.47402  18.838604    7.44684    1.65929
81862.741801 11401.502123     11.5
min     2017.0      8.0      1.0      0.0          57993.0
0.0      0.429      0.151      0.117      0.0576      0.0455      0.0224
4.3      0.133      0.0

```

25%	2017.0	9.0	7.0	588.75		57999.0	
21525.0		11.7	0.279	0.186	0.0984	0.0597	0.0263
22200.0		552.0	5.75				
50%	2017.0	9.0	13.0	1177.5		58005.0	
43050.0		55.4	0.632	0.376	0.146	0.0866	0.0363
68600.0		4290.0	11.5				
75%	2017.0	9.0	19.0	1766.25		58011.0	
64575.0		560.0	107.0	35.8	0.99	0.249	0.067
143000.0		14100.0	17.25				
max	2017.0	9.0	31.0	2355.0		58017.0	
86100.0		32500.0	2890.0	1330.0	399.0	191.0	61.5
259000.0		110000.0	23.0				
std	0.0	0.366632	8.775483	692.481902		7.211603	
24943.113498		1625.968447	390.921861	222.362627	67.265595	28.620524	7.15889
65728.005824		16920.617446	6.922667				

	minute	ts
count	7200.0	7200
mean	27.5	2017-09-09 11:57:30
min	0.0	2017-08-28 00:00:00
25%	13.75	2017-09-03 05:58:45
50%	27.5	2017-09-09 11:57:30
75%	41.25	2017-09-15 17:56:15
max	55.0	2017-09-21 23:55:00
std	17.261461	NaN

3.3: ()
(, ..), ,

5 4. '

: 5 , . , . .
:
1. A: 5 → . — resample + .
2. C: . , ts — merge.
3. B: → . , 24 .

5.0.1 4.1

A . :

```
[17]: #
A = A.set_index("ts")
A_h = A.resample("1h").agg(["mean", "max"])
A = A.reset_index()
```

A_h

[17]:

Julian Day		year		month		day		HHMM		Modified	
P >10		P >30		\		P > 1				P > 5	
						</					

```

ts
2017-08-28 00:00:00 0.077583 0.178 0.040692 0.0793 20941.666667 23600.0
1011.583333 1180.0 0.0 0 27.5 55
2017-08-28 01:00:00 0.0587 0.0798 0.02995 0.0471 18091.666667 20600.0
713.0 907.0 1.0 1 27.5 55
2017-08-28 02:00:00 0.069642 0.111 0.035025 0.0595 14108.333333 15000.0
380.833333 475.0 2.0 2 27.5 55
2017-08-28 03:00:00 0.068092 0.139 0.035517 0.0818 12766.666667 13100.0
267.916667 284.0 3.0 3 27.5 55
2017-08-28 04:00:00 0.07285 0.142 0.039708 0.0694 11333.333333 12300.0
203.0 234.0 4.0 4 27.5 55
...
...
...
2017-09-21 19:00:00 0.0743 0.105 0.040192 0.0687 180250.0 186000.0
65750.0 66800.0 19.0 19 27.5 55
2017-09-21 20:00:00 0.069983 0.105 0.039692 0.0737 178750.0 186000.0
63175.0 70500.0 20.0 20 27.5 55
2017-09-21 21:00:00 0.077467 0.147 0.033417 0.0524 175833.333333 177000.0
58350.0 60300.0 21.0 21 27.5 55
2017-09-21 22:00:00 0.070633 0.108 0.0396 0.0798 167750.0 176000.0
49366.666667 54400.0 22.0 22 27.5 55
2017-09-21 23:00:00 0.078108 0.111 0.045125 0.0733 150083.333333 160000.0
37525.0 44400.0 23.0 23 27.5 55

```

[600 rows x 32 columns]

5.0.2 4.2

(, ,).

, , .

: ?

```

[18]: #
A = A.set_index("ts")
A_h_my = A["P >50"].iloc[:24*12].resample("1h").agg(["median", "mean"])
A = A.reset_index()
A_h_my

```

```

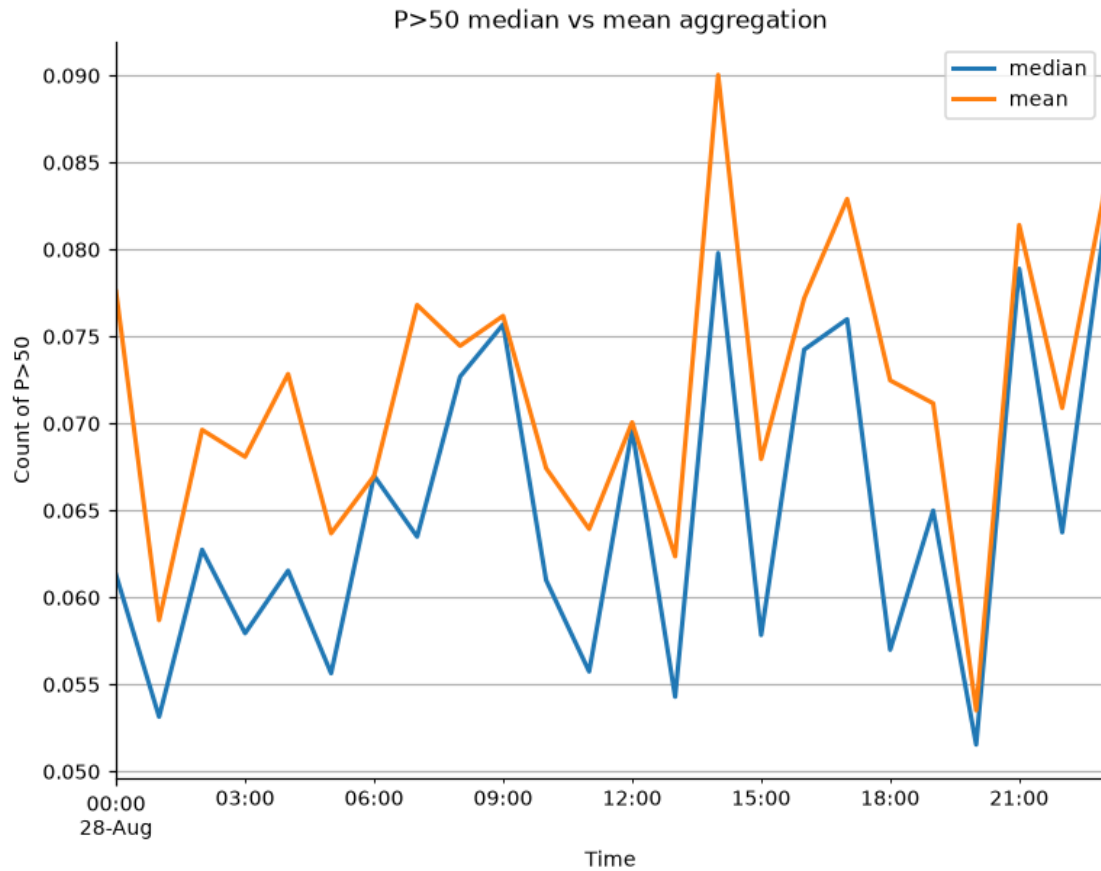
[18]:          median    mean
ts
2017-08-28 00:00:00 0.0613 0.077583
2017-08-28 01:00:00 0.05315 0.0587
2017-08-28 02:00:00 0.06275 0.069642
2017-08-28 03:00:00 0.05795 0.068092
2017-08-28 04:00:00 0.06155 0.07285
2017-08-28 05:00:00 0.05565 0.063692
2017-08-28 06:00:00 0.067 0.066992

```

2017-08-28 07:00:00	0.0635	0.076817
2017-08-28 08:00:00	0.0727	0.074467
2017-08-28 09:00:00	0.0757	0.076192
2017-08-28 10:00:00	0.061	0.067433
2017-08-28 11:00:00	0.05575	0.063942
2017-08-28 12:00:00	0.06965	0.070075
2017-08-28 13:00:00	0.0543	0.062367
2017-08-28 14:00:00	0.0798	0.090042
2017-08-28 15:00:00	0.05785	0.067958
2017-08-28 16:00:00	0.07425	0.0772
2017-08-28 17:00:00	0.076	0.082908
2017-08-28 18:00:00	0.057	0.072483
2017-08-28 19:00:00	0.065	0.071167
2017-08-28 20:00:00	0.05155	0.053517
2017-08-28 21:00:00	0.0789	0.081417
2017-08-28 22:00:00	0.06375	0.0709
2017-08-28 23:00:00	0.08165	0.0835

```
[19]: A_h_my.plot(figsize=[8,6], linewidth=2)

plt.title("P>50 median vs mean aggregation")
plt.xlabel('Time')
plt.ylabel('Count of P>50')
plt.grid(True)
plt.show()
```



4.2: ()

5- (). ,

5.0.3 4.3

, — M. , .
:
how='outer'). , , , (how='inner'

```
[20]: #
B = B.set_index("ts")
B_h = B.resample("1h").ffill()
B = B.reset_index()

C_h = C.set_index("ts")
M = pd.concat([A_h, B_h, C_h], axis=1, join="outer").fillna(0)
M = M[['SPEED', 'DENSITY', 'Radio Flux 10.7'] + [i for i in M.columns if i[0].startswith('P') or i[0].startswith('E')]]
```

[20] :

	SPEED	DENSITY	Radio Flux 10.7	(P > 1, mean)	(P > 1, max)
	(P > 5, mean)	(P > 5, max)	(P >10, mean)	(P >10, max)	(P >30, mean)
	(P >30, max)	(P >50, mean)	(P >50, max)	\	

ts

2017-08-28 00:00:00	285	3.0	82	9.778333	
19.4	0.324917	0.69	0.192917	0.314	0.09885
0.224	0.077583	0.178			
2017-08-28 01:00:00	305	3.4	82	7.994167	
13.6	0.291667	0.512	0.156583	0.259	0.072583
0.0919	0.0587	0.0798			
2017-08-28 02:00:00	309	1.7	82	6.099167	
10.8	0.283	0.479	0.177667	0.321	0.08445
0.121	0.069642	0.111			
2017-08-28 03:00:00	304	2.0	82	4.7525	
7.76	0.236833	0.329	0.166333	0.244	0.087658
0.18	0.068092	0.139			
2017-08-28 04:00:00	302	2.3	82	4.158333	
6.62	0.334917	0.539	0.208333	0.309	0.0848
0.153	0.07285	0.142			

...

...

...

2017-09-21	20:00:00	379	1.8	0	42.1025	
78.9	0.3975		0.833	0.226833	0.429	0.089325
0.138	0.069983		0.105			
2017-09-21	21:00:00	380	1.8	0	50.783333	
71.7	0.48925		0.788	0.317583	0.501	0.121358
0.204	0.077467		0.147			
2017-09-21	22:00:00	387	1.1	0	39.0	
48.8	0.35		0.536	0.20025	0.31	0.094358
0.143	0.070633		0.108			
2017-09-21	23:00:00	380	1.0	0	28.825	
42.1	0.465		0.861	0.257833	0.506	0.113575
0.168	0.078108		0.111			
2017-09-22	00:00:00	381	1.0	0	0.0	
0.0	0.0		0.0	0.0	0.0	0.0
0.0	0.0		0.0			

	(P>100, mean)	(P>100, max)	(E>0.8, mean)	(E>0.8, max)
(E>2.0, mean)	0.0000	0.0000	0.0000	0.0000
(E>2.0, max)	0.0000	0.0000	0.0000	0.0000

ts

2017-08-28 00:00:00	0.040692	0.0793	20941.666667	23600.0
1011.583333	1180.0			
2017-08-28 01:00:00	0.02995	0.0471	18091.666667	20600.0
713.0	907.0			

```
[601 rows x 19 columns]
```

inner (601 577) , dataframe, outer - , NA. outer,

P>10 :

		10	
	10		1000
1000			

M
:
.

NOAA.

30

```
M = M.reset_index()
M
```

	ts	SPEED	DENSITY	Radio Flux 10.7	(P > 1, mean)	(P > 1, max)
	(P > 5, mean)	(P > 5, max)	(P >10, mean)	(P >10, max)	(P >30, mean)	(P > 1, max)
	(P >30, max)	(P >50, mean)	\			
0	2017-08-28 00:00:00	285	3.0	82	9.778333	
19.4	0.324917	0.69	0.192917	0.314	0.09885	
0.224	0.077583					
1	2017-08-28 01:00:00	305	3.4	82	7.994167	
13.6	0.291667	0.512	0.156583	0.259	0.072583	
0.0919	0.0587					
2	2017-08-28 02:00:00	309	1.7	82	6.099167	
10.8	0.283	0.479	0.177667	0.321	0.08445	
0.121	0.069642					
3	2017-08-28 03:00:00	304	2.0	82	4.7525	
7.76	0.236833	0.329	0.166333	0.244	0.087658	
0.18	0.068092					
4	2017-08-28 04:00:00	302	2.3	82	4.158333	
6.62	0.334917	0.539	0.208333	0.309	0.0848	
0.153	0.07285					
..	...	...	...	...	...	
...	...	...	...	...	...	
...	...					
596	2017-09-21 20:00:00	379	1.8	0	42.1025	
78.9	0.3975	0.833	0.226833	0.429	0.089325	
0.138	0.069983					
597	2017-09-21 21:00:00	380	1.8	0	50.783333	
71.7	0.48925	0.788	0.317583	0.501	0.121358	
0.204	0.077467					
598	2017-09-21 22:00:00	387	1.1	0	39.0	
48.8	0.35	0.536	0.20025	0.31	0.094358	
0.143	0.070633					
599	2017-09-21 23:00:00	380	1.0	0	28.825	
42.1	0.465	0.861	0.257833	0.506	0.113575	
0.168	0.078108					
600	2017-09-22 00:00:00	381	1.0	0	0.0	
0.0	0.0	0.0	0.0	0.0	0.0	
0.0	0.0					
	(P >50, max)	(P>100, mean)	(P>100, max)	(E>0.8, mean)	(E>0.8, max)	
	(E>2.0, mean)	(E>2.0, max)				
0	0.178	0.040692	0.0793	20941.666667	23600.0	
1011.583333	1180.0					
1	0.0798	0.02995	0.0471	18091.666667	20600.0	
713.0	907.0					
2	0.111	0.035025	0.0595	14108.333333	15000.0	

380.833333		475.0			
3	0.139	0.035517	0.0818	12766.666667	13100.0
267.916667		284.0			
4	0.142	0.039708	0.0694	11333.333333	12300.0
203.0	234.0				
..	...	...	...	...	...
...	...	...			
596	0.105	0.039692	0.0737	178750.0	186000.0
63175.0	70500.0				
597	0.147	0.033417	0.0524	175833.333333	177000.0
58350.0	60300.0				
598	0.108	0.0396	0.0798	167750.0	176000.0
49366.666667	54400.0				
599	0.111	0.045125	0.0733	150083.333333	160000.0
37525.0	44400.0				
600	0.0	0.0	0.0	0.0	0.0
0.0	0.0				

[601 rows x 21 columns]

4.

```
[23]: assert M is not None, 'M is None'
assert 'ts' in M.columns, 'M does not have a time series'
assert 'id' in M.columns, 'M does not have an identifier'
assert 590 <= len(M) <= 605, 'M has {} rows, expected between 590 and 605'.format(len(M))
print('M has {} rows'.format(len(M)))
```

4 : 601

6 5. ' — '

, « ».

:

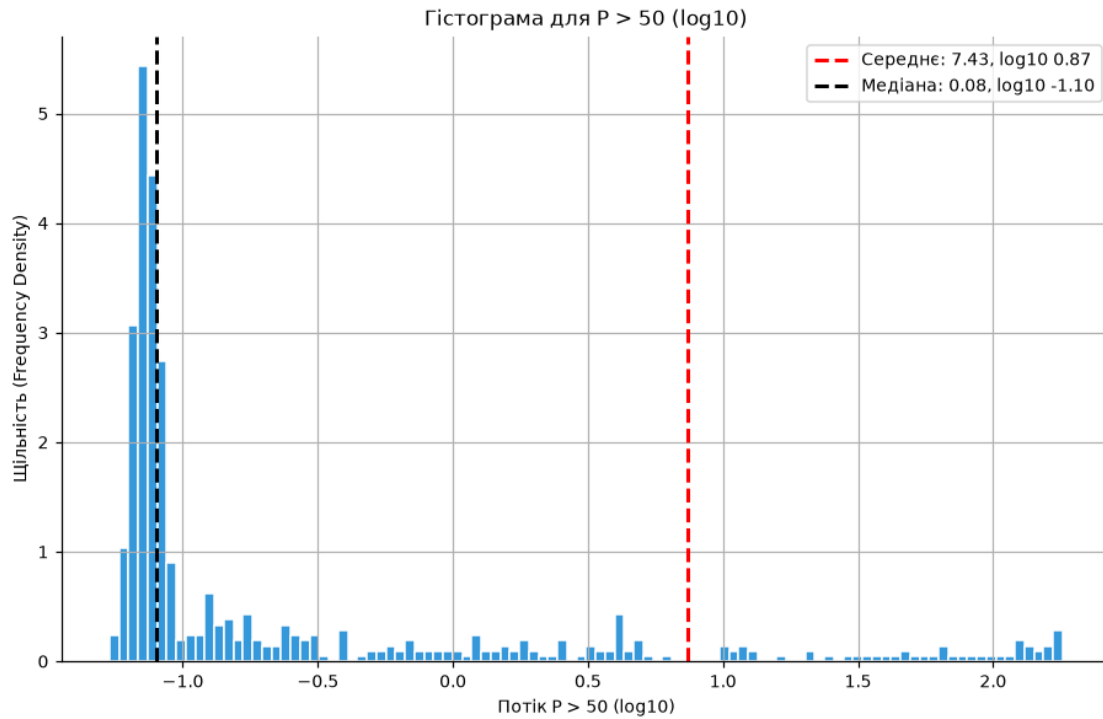
- , ;
 - ;
 - , — . « », «
- 60, ».
- « , ' ».
- , , « ».

6.0.1 1. ?

().

```
(np.log10) , .
:
```

```
[24]: #
hist_data = M.loc[M[('P >50', 'mean')]>0, [('P >50', 'mean')]].dropna()
plt.figure(figsize=(10, 6))
plt.hist(
    np.log10(hist_data),
    bins=100,
    color="#3498db",
    edgecolor="white",
    alpha=1,
    density=True,
)
plt.title(
    "      P > 50 (log10)"
)
plt.axvline(
    np.log10(M[('P >50', 'mean')].mean()),
    color="red",
    linestyle="--",
    linewidth=2,
    label=f"      : {M[('P >50', 'mean')].mean():.2f}, log10 {np.log10(M[('P_
↪>50', 'mean')].mean():.2f}",
)
plt.axvline(
    np.log10(M[('P >50', 'mean')].median()),
    color="black",
    linestyle="--",
    linewidth=2,
    label=f"      : {M[('P >50', 'mean')].median():.2f}, log10 {np.log10(M[('P_
↪>50', 'mean')].median():.2f}",
)
plt.xlabel("      P > 50 (log10)")
plt.ylabel("      (Frequency Density)")
plt.legend()
plt.grid(True)
plt.show()
```



: P > 50 $10^{-1.2}$, (7.43) (0.08),

6.0.2 2. ?

(boxplot)

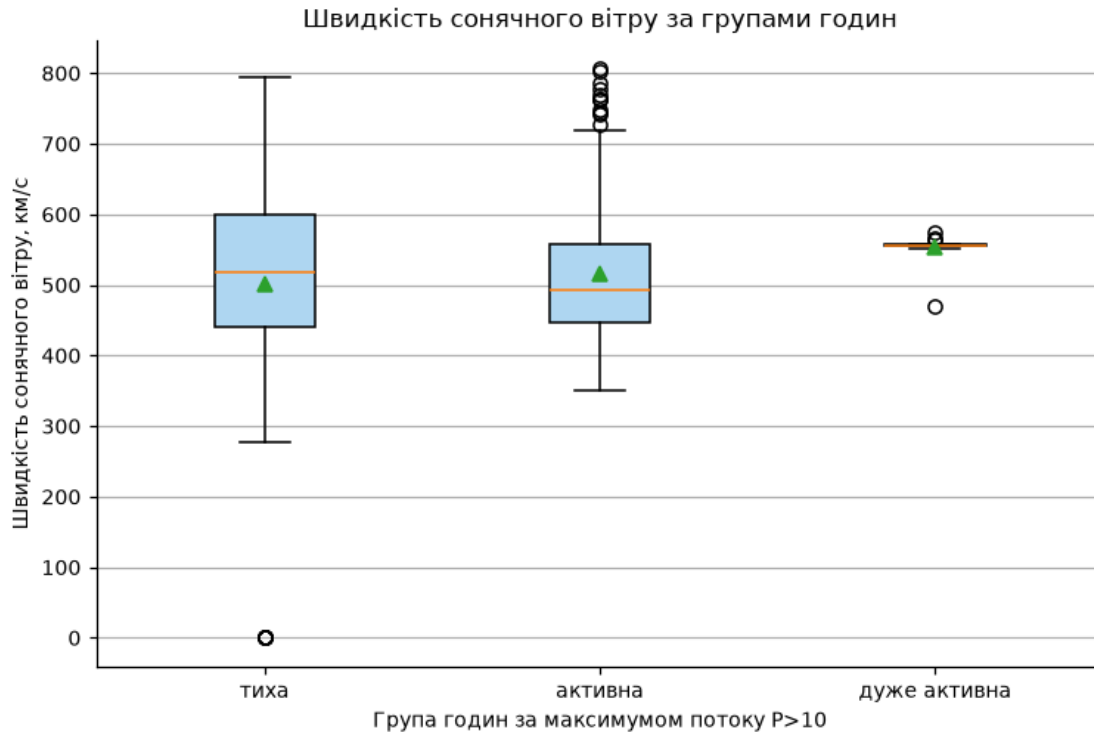
4.4.

```
[25]: #
groups = [' ', ' ', ' ']

data = [M.loc[M[' '] == g, 'SPEED'].dropna() for g in groups]

plt.figure(figsize=(8, 5))
bp = plt.boxplot(data, tick_labels=groups, showmeans=True, patch_artist=True)
for box in bp['boxes']:
    box.set_facecolor('#aed6f1')
plt.title(' ')
plt.xlabel(' P>10')
plt.ylabel(' , / ')
plt.grid(True, axis='y')
plt.show()

M.groupby(' ')['SPEED'].describe()
```



[25]:

	count	mean	std	min	25%	50%	75%	max
171.0	515.05848	103.401087	350.0	447.5	492.0	557.0	806.0	
18.0	552.944444	21.631873	469.0	555.0	556.0	557.0	575.0	
412.0	501.007282	134.982607	0.0	440.25	517.5	599.0	795.0	

: (~500, 556 “ ”)

6.0.3 3. , ?

```
[26]: #
corr = M[['SPEED', ('P >10', 'max')]].corr().to_numpy()
print(f'Correlation: {corr[0,1]}')

plt.figure(figsize=(7,5))
plt.scatter(
    M["SPEED"], M[('P >10', 'max')], color="royalblue", alpha=0.7,
    edgecolors="k"
)

plt.xlabel("SPEED", fontsize=11)
```

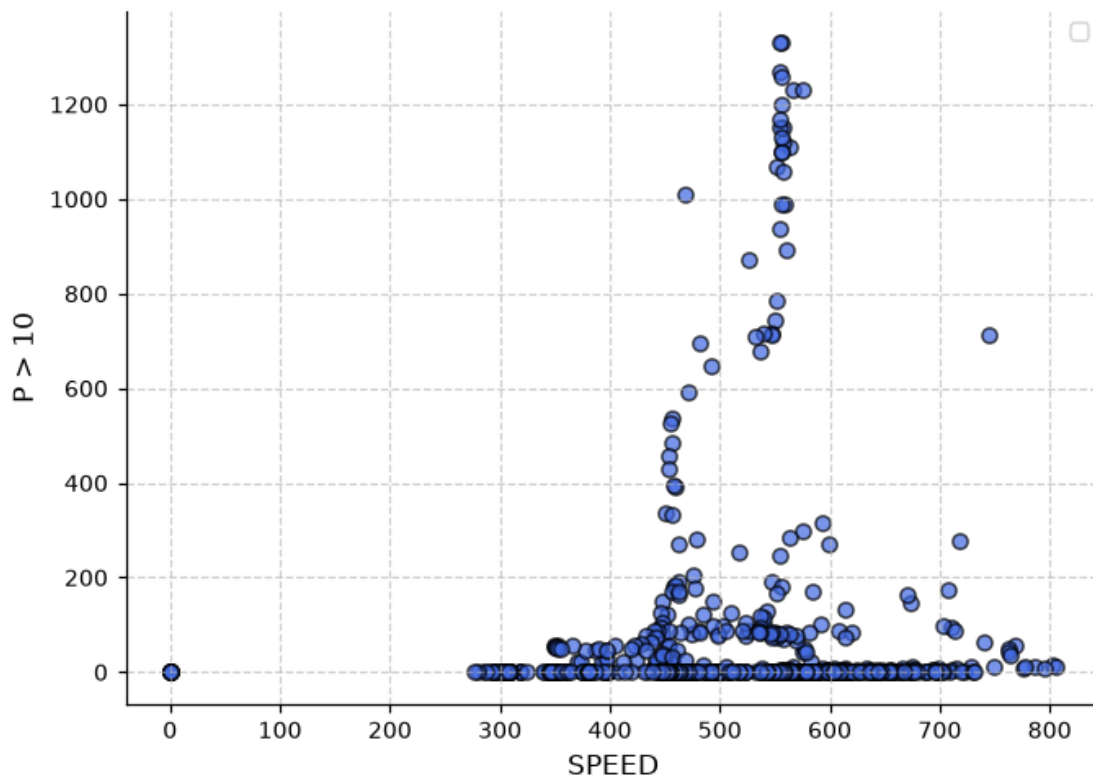
```
plt.ylabel("P > 10", fontsize=11)
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()

plt.show()
```

Correlation: 0.08856160778553436

/tmp/ipykernel_10677/1323394169.py:13: UserWarning: No artists with labels found to put in legend. Note that artists whose label start with an underscore are ignored when legend() is called with no argument.

```
plt.legend()
```



```
:
400-500, P>10 ( 0.089), P>10
```

6.0.4 4. ?

```
[27]: #
groups = [' ', ' ', ' ', ' ']
counts = M[' '].value_counts().reindex(groups, fill_value=0)
```

```
percentage = counts / counts.sum() * 100

plt.figure(figsize=(7, 5))
bars = plt.bar(counts.index, counts.values, color=['#85c1e9', '#f5b041', '#e74c3c'])

for bar, n, p in zip(bars, counts.values, percentage.values):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height(),
             f'{n} \n({p:.1f}%)', ha='center', va='bottom', fontsize=9)

plt.title(' ')
plt.xlabel(' ')
plt.ylabel(' ')
plt.grid(True, axis='y')
plt.show()
```



: 68.6% , 28.5 3

6.0.5 5. ?

P>10

1.

```
[28]: #
series = M[('P >10', 'max')]

plt.figure(figsize=(12, 4))
plt.plot(M['ts'], series, color='#2a78d6', linewidth=1)
plt.yscale('log')
plt.title('          P>10          ')
plt.xlabel(' ')
plt.ylabel('          P>10')
plt.grid(True, which='both', alpha=0.3)
plt.show()
```



```
[29]: #
series = M[('P >10', 'max')].where(M['ts'] > pd.Timestamp("2017-09-09 00:00:
↪00"))

plt.figure(figsize=(12, 4))
plt.plot(M['ts'], series, color='#2a78d6', linewidth=1)
plt.yscale('log')
plt.title('          P>10          ( 9      )')
plt.xlabel(' ')
plt.ylabel('          P>10')
plt.grid(True, which='both', alpha=0.3)
plt.show()
```



: P>10 5 9 10 15 . , 1000,
,

7 6.

```
[30]: = ('P >10', 'max') # <- P>10

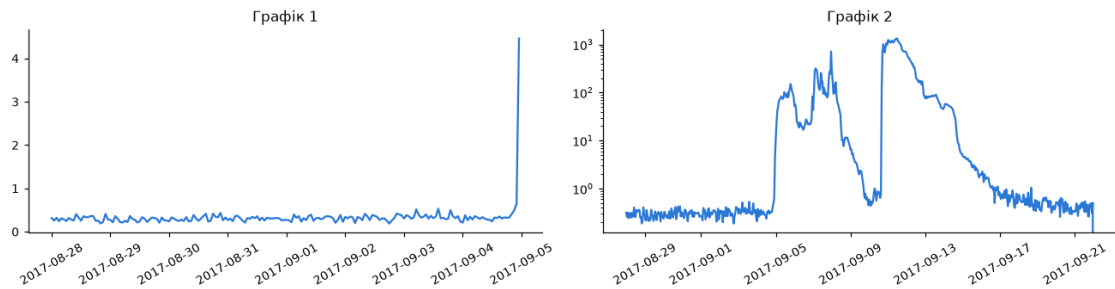
      = M[(M['ts'] >= '2017-08-28') & (M['ts'] < '2017-09-05')]

fig, ax = plt.subplots(1, 2, figsize=(12, 3.2))

ax[0].plot( ['ts'], [ ], color='#2a78d6')
ax[0].set_title(' 1')
ax[0].tick_params(axis='x', rotation=30)

ax[1].plot(M['ts'], M[ ], color='#2a78d6')
ax[1].set_yscale('log')
ax[1].set_title(' 2')
ax[1].tick_params(axis='x', rotation=30)

fig.tight_layout()
plt.show()
```



7.0.1 6.1

- :
1. 2017 , ?
 2. , ?
 3. ? — .
- .

6.1:

1. : 5 , -
2. : 5 9 10 15 -
3. : log scale , -

7.0.2 6.2

,

.

,

.

```
[31]: #
series = M[('P >10', 'max')]

plt.figure(figsize=(12, 4))
plt.plot(M['ts'], series, color='#2a78d6', linewidth=1)
plt.yscale('log')
plt.axhline(10, color='gray', linestyle='--', linewidth=1)
plt.axhline(1000, color='gray', linestyle='--', linewidth=1)
plt.title(' ')
plt.xlabel(' ')
plt.ylabel(' P>10 ')
plt.grid(True, which='both', alpha=0.3)
plt.show()
```



6.2: log scale, , .
10 1000, NOAA,

8 7.

8.0.1 7.1

, .
— , « », , «
2000 , ».

, .
1: log scale,
, ,
2: 1000 (),
,

3: ,

8.0.2 7.2 —

, : ,

```
[32]: #
M = M.sort_values('ts').reset_index(drop=True)

active = M[''].isin(['', ''])
runs = (active != active.shift()).cumsum()

lengths = active.groupby(runs).sum()
```

```

starts = M.loc[active, 'ts'].groupby(runs).min()
ends = M.loc[active, 'ts'].groupby(runs).max()

active_runs = pd.DataFrame({'': lengths, '': starts, '': ends})
active_runs = active_runs[active_runs[''] > 0].sort_values(' ',
↪ascending=False)

print('':', active_runs[''].iloc[0])
print(' :', active_runs[''].iloc[0], ' :', active_runs[''].iloc[0])
print()
print(active_runs)

```

```

: 98
: 2017-09-10 16:00:00 : 2017-09-14 17:00:00

```

```

6      98 2017-09-10 16:00:00 2017-09-14 17:00:00
2      87 2017-09-05 00:00:00 2017-09-08 14:00:00
4       4 2017-09-08 17:00:00 2017-09-08 20:00:00

```

7.2: “ ” “ ” , 98 , 10 16:00 14
17:00

9 8.

. ,
 .
 , : , — ,
 .
 : , . , ,
 , . , — .
 , , .

9.0.1 8.1

. :
 • ' ' — ;
 • ' ' — , ;
 • ' , ' — , , ;
 • ' , ' — , ;
 • ' , ' — .
 , : , .

```
[33]: _ = 'Gemini, Kilo code (deepseek v4.1 flash)' # . 'ChatGPT' '

↳

= {
    '1-2,': ' ',
    '3,': ' ',
    '4,': ' ',
    '5,': ' ',
    '5,': ' ',
    '6-7,': ' ',
}

#
#
- = ' ',
( , , , - ) \
,

# True :
#
= True
```

```
[34]: # ---
= {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}

assert _ .strip(), ' « »'
= [k for k, v in .items() if not str(v).strip()]
assert not , ' : %s' % '; '.join( )
= {v for v in .values() if v not in }
assert not , ' : %s. ' % ', '.join(sorted( ))
assert len( _ .strip()) >= 120, (' ',
'( %d , 120)'
% len( _ .strip()))

assert is True, ' ,

print(' ')
print('=' * 78)
print('%-46s %s' % (' :', ))
print('%-46s %s' % (' :', _ ))
print('-' * 78)
for k, v in .items():
    print('%-46s %s' % (k + ':', v))
print('-' * 78)
print(' :')
print( _ )
print('=' * 78)
print(' , .')
```

```

=====
:
: Gemini, Kilo code (deepseek v4.1
flash)
-----
1-2, :
3, :
4, ' :
5, : ,
5, :
6-7, : ,
-----
:
, , - ) , ( ,
, , - ) ,
=====
, .

```

10 9.

, № 3 № 4 — .

```
[35]: M.to_csv('master_hourly.csv', index=False)
print(' :', M.shape)
```

: (601, 21)

```
[36]: # --- -----
      = {
      ' : bool( ) and bool( ),
      ' A (7200 )': len(A) == 7200,
      ' B (25 )': len(B) == 25,
      ' C (601 )': len(C) == 601,
      ' , ': M is not None and 590 <= len(M) <= 605,
      ' « » ': ' ' in M.columns,
      ' 2017 ': int(pd.to_datetime(M['ts']).dt.year.min()) == 2017,
      ' ': bool( _ .strip()) and is True,
      }
for k, v in .items():
    print(('OK ' if v else ' ') + ' ' + k)

if all( .values()):
    print('\n .')
    print(' , ,')
```

```
print('      Kernel → Restart & Run All      .')
```

OK

OK A (7200)

OK B (25)

OK C (601)

OK ,

OK « »

OK 2017

OK

.

,

Kernel → Restart & Run All .